DIGITAL HEROES
ENGINEERING · DEVELOPER PROGRAM

Full Stack Developer Trial Task

The Builder's Handbook

Design, build, ship, and open-source one real product.
A complete playbook — from choosing an idea to a deployed, documented, portfolio-grade application that proves you can ship like a senior engineer.

TIMEBOX	STACK	OUTPUT	BUILT WITH
7–14 days	Your choice · full-stack	Deployed app + GitHub	Claude in Antigravity

A Digital Heroes program · Founded by Prasun Anand

Contents

SECTION · 01

Introduction & Why This Task Exists

> **From Prasun Anand · Founder of Digital Heroes**
> I'm not impressed by how many frameworks you can name. I'm impressed when someone takes a real problem and ships a real product a real person would use. Do that here, and you have my full attention.

This is not a quiz, a take-home puzzle, or a whiteboard exercise. It is a **build**. You will conceive, design, engineer, deploy, document, and open-source one real full-stack product — the kind a company would actually pay for. The task exists because the only reliable signal of an engineer is a shipped product you can click, break, and read the code behind.

Anyone can pass an interview. Far fewer can take an idea from an empty repository to a polished, deployed application with authentication, real data, thoughtful UX, clean commits, and documentation a stranger could follow. That gap is exactly what this handbook closes. Follow it and you will finish with more than a trial submission — you will own a portfolio piece that keeps earning you opportunities long after this is over.

> **What you're really being measured on**
> Judgment. Taste. Whether you sweat the details a user will notice and a reviewer will respect. A todo app built with obsessive craft beats an ERP built carelessly. **Depth over breadth, finish over features.**

SECTION · 02

Objectives — What You Will Prove

By the end, your single submission should demonstrate — with evidence, not claims — that you can:

- **Own a product end to end** — from a blank folder to a live URL anyone can use.
- **Model real data** — schemas, relationships, and state that hold up under real use.
- **Design an interface with taste** — clean, fast, accessible, and consistent.
- **Write code a senior would approve** — readable, structured, and safe to change.
- **Ship and operate** — deploy, configure env, handle errors, and keep it up.
- **Communicate** — a README, docs, and a case study that make your work legible.
- **Work with AI like a pro** — direct Claude in Antigravity as a force multiplier, not a crutch.

THE LENS · 03

Build Through the End User's Eyes

Before and during every decision, hold your work against two questions. They matter more than any framework you pick.

- **Is my UI instantly obvious?** Could the person who will actually use this figure it out with no manual and no training — on their very first try?
- **Is the functionality genuinely useful?** If this shipped to the real world tomorrow, would the automation actually save real people real time, money, and wasted effort?

> **This is the craft we're looking for**
> Judge every screen and every feature against those two questions. If the honest answer is yes, the software is good enough by default. That instinct — building for the human on the other side, not for the reviewer — is exactly what we want to see.

SECTION · 04

60+ Project Ideas

Pick something **CRUD-rich and dashboard-heavy** — it naturally exercises auth, roles, data modeling, search, and analytics. Below are 60+ starting points across nine verticals. Steal one, narrow it, and make it yours.

CRM & Sales

Product	One-line pitch
PipelineIQ	Kanban deal pipeline with weighted forecasting for B2B sales teams.
TerritoryMap	Geo-assign accounts and quotas to reps with coverage analytics.
QuoteForge	Configure-price-quote tool generating branded PDF proposals for closers.
ChurnRadar	Flags at-risk accounts from usage signals for CSMs.
CommissionDesk	Automates rep commission math and payout approvals for RevOps.
LeadRouter	Round-robins inbound leads by rules with SLA tracking.
CallSheet	Logs sales calls with next-step reminders for reps.

HR & Recruiting (ATS/HRMS)

Product	One-line pitch
HireTrack	Applicant pipeline with scorecards and interview scheduling for recruiters.
OnboardFlow	Checklist-driven employee onboarding with document e-signatures for HR.
PeoplePulse	Anonymous engagement surveys with trend dashboards for people teams.
LeaveLedger	PTO requests, approvals, and balance tracking for managers.
ReviewCycle	360-degree performance reviews with goal tracking for teams.
ReferralHub	Employee referral program with reward tracking for talent teams.
OrgChartly	Live org chart and headcount planning for HR ops.
SkillMatrix	Maps team competencies against role gaps for managers.

Operations & Logistics (ERP/WMS/Inventory)

Product	One-line pitch
StockPilot	Multi-warehouse inventory with low-stock alerts for operations teams.
RouteBoard	Delivery route planning with driver assignment for dispatchers.
OrderForge	Purchase orders and supplier approvals for procurement teams.
AssetVault	Tracks equipment assignments and maintenance schedules for facilities.
ShipTrace	Shipment tracking with milestone alerts for logistics coordinators.
BOMBuilder	Bill-of-materials and production scheduling for small manufacturers.
WarehouseMap	Bin-location picking and cycle counts for warehouse staff.

Finance & Billing

Product	One-line pitch
InvoiceLoop	Recurring invoices with dunning and payment tracking for freelancers.
ExpenseDesk	Receipt capture, approvals, and reimbursement tracking for finance teams.
SubMeter	Subscription billing with usage metering and MRR dashboards.
BudgetBoard	Budgets versus actuals with variance alerts for controllers.
ReconcileIQ	Matches bank transactions to ledger entries for bookkeepers.
CapStack	Cap table and equity vesting tracking for founders.
PayrollLite	Runs contractor payroll with tax summaries for small businesses.

Healthcare

Product	One-line pitch
ClinicQueue	Patient scheduling and check-in with wait-time dashboards for clinics.
RxTracker	Medication adherence tracking with refill reminders for care teams.
ChartVault	Patient records with visit notes and role-based access.
TeleSlot	Telehealth booking with intake forms for private practices.
ClaimDesk	Insurance claim submission and status tracking for billers.
VitalsLog	Remote vitals logging with threshold alerts for nurses.
ConsentFlow	Digital patient consent forms with audit trails for clinics.

Education / LMS

Product	One-line pitch
CourseForge	Build lessons, quizzes, and progress tracking for course instructors.
GradeBook	Assignment submission and rubric-based grading for teachers.
CohortHub	Cohort-based course management with discussion boards for creators.
QuizArena	Timed quizzes with auto-grading and leaderboards for schools.
AttendLog	Class attendance and participation tracking for administrators.
TutorMatch	Books tutoring sessions and tracks progress for learning centers.
CertMint	Issues verifiable course certificates with completion tracking for academies.

Developer Tools

Product	One-line pitch
StatusForge	Public status pages with incident timelines for engineering teams.
FlagDeck	Feature flags and gradual rollouts with audit logs.
CronWatch	Monitors scheduled jobs and alerts on missed runs.
APIMeter	API key management with rate limits and usage analytics.
ErrorNest	Aggregates app errors with grouping and alert rules.
ChangeLoggr	Publishes product changelogs with subscriber notifications for teams.
SchemaDiff	Tracks database schema changes with migration approvals for teams.
UptimePing	Endpoint uptime monitoring with SLA reports for ops.

Marketing & Content

Product	One-line pitch
ContentQ	Editorial calendar with approval workflows for content teams.
LinkVault	Branded short links with click analytics for marketers.
CampaignDesk	Multi-channel campaign planning with UTM tracking for growth teams.
InfluencerCRM	Manages creator outreach and deliverables for brand marketers.
SurveyStack	Builds surveys with response analytics for market researchers.
AssetLibrary	Digital asset management with brand approvals for agencies.
LeadMagnetHub	Gated content delivery with lead capture for marketers.

AI-Native Apps

Product	One-line pitch
DocQuery	Chat with uploaded documents using retrieval for knowledge workers.
MeetScribe	Transcribes meetings and extracts action items for teams.
SupportCopilot	Drafts support replies from your knowledge base for agents.
ResumeRank	Screens and scores applicants against job descriptions for recruiters.
PromptDesk	Version and test LLM prompts with eval dashboards.
ReviewMiner	Summarizes customer reviews into themes and sentiment for PMs.
ContractLens	Extracts clauses and risks from contracts for legal teams.
VoiceNotes	Turns voice memos into structured notes for founders.

SECTION · 05

Researching the World's Best Products

Before you write a line of code, spend a few hours studying how the best products in your category actually work. You are reverse-engineering good decisions.

Pick 2-3 category leaders. Choose the acknowledged leader plus one fast-moving challenger and one adjacent outlier (e.g. Linear, Height, Notion). Filter by G2/Product Hunt traction and a real free tier so you can actually get inside the product.

Sign up and time the onboarding. Create a real account on each with a stopwatch running. Log every screen from landing to activation, and mark the 'aha' moment plus every place friction spikes.

Screenshot the core flows end-to-end. Capture the 3-4 critical paths (create, edit, share, delete) at 1x and mobile widths using CleanShot/Shottr or Chrome DevTools device mode. Drop them into a Figma board tagged per product.

Reverse-engineer the data model from the UI. For each list and detail screen, infer entities, fields, enums, and relationships their forms and filters expose. Sketch it as an ERD before you write a single migration.

Dissect empty states and micro-interactions. Screen-record at 60fps and step through hover, focus, loading skeletons, optimistic updates, and toast timing. Write down the easing curve and duration in ms.

Read the changelog and docs. Skim 12-18 months of changelog plus the help center and API reference. See what they cut, what they doubled down on, and the exact vocabulary they teach users.

What to extract

- Onboarding: steps to first value, mandatory vs skippable fields, sample-data seeding, and total time-to-activation.
- Navigation & IA: top-level item count, sidebar vs topbar, hierarchy depth, breadcrumbs, and Cmd+K palette coverage.
- Dashboard layout: what loads above the fold for a returning user, widget grid vs single focus, and information density.
- Table/list patterns: sort/filter/group controls, inline edit, bulk actions, pagination vs infinite scroll, sticky headers, density toggle.
- Forms: field grouping, inline-validation timing, autosave vs explicit save, tab order, and the tone of error copy.
- Empty states: first-run vs zero-results vs error variants, and how illustration + CTA + template offers are paired.
- Pricing page: tier count, anchor pricing, feature gating, whether annual is the default toggle, and per-tier CTA copy.
- Settings: org vs personal split, danger-zone treatment, in-settings search, and save patterns.
- Notifications: in-app vs email vs push, badge/toast/inbox split, batching, and granularity of preferences.
- Keyboard shortcuts: '?' overlay discoverability, palette coverage of core actions, and whether they mirror Linear/Superhuman conventions.

> **Inspiration vs. copying**
> Copy the interaction model and information hierarchy, never the pixels — a Cmd+K palette or a filter dropdown is a shared pattern, their exact layout and spacing are theirs. Never lift brand assets: logos, color tokens, marketing copy, illustrations, and icon sets are trademark/copyright-protected, and ripping them is infringement, not inspiration. Redraw everything in your own identity — original naming, voice, palette, and empty-state copy; if a screen is recognizable as theirs, you crossed the line. Keep a private research doc noting what came from where so you can defend originality later; convergent UX is fine, verbatim clones are not.

WORKFLOW · 06

AI Development Workflow — Claude in Google Antigravity

Antigravity gives you Claude as an agentic pair. The difference between a mediocre and a magic result is entirely in how you drive it. Treat Claude like a brilliant, fast senior engineer who needs a clear brief and a firm reviewer.

Spec before a single line. Write acceptance criteria, data shapes, and edge cases into a plan.md and make Claude confirm it before coding — ambiguity killed in prose is 10x cheaper than in a diff.

Ship in milestones, not monoliths. Cap each turn at one vertical slice — schema, then one endpoint, then one screen. A 500-line generation you can't fully review is a liability, not velocity.

Schema and types first, features second. Lock the migration and TypeScript types before any feature code so every downstream generation inherits the contract instead of inventing its own.

Demand tests in the same turn. Ask for the failing test alongside the implementation, then watch it go green — a feature Claude wrote and Claude tested still needs a test you can read and run.

Review every diff — never blind-paste. Read each hunk before accepting. Claude confidently invents API signatures, and the bug it hands you costs more than the code it saved.

Feed it the stack trace, not a summary. Paste the full error, the exact failing command, and the relevant logs — Claude self-fixes from raw evidence far better than from your paraphrase of the symptom.

Keep the context window tight. Start a fresh thread per milestone and reference files by path — a bloated 100k-token session degrades reasoning and buries the one instruction that matters.

Ask 'is there a more elegant way?'. After a working solution, force one refactor pass. The first draft ships; the second draft is the one a staff engineer approves.

Copy-ready prompts

Planning / spec-first

```
Before writing any code, produce a plan.md for <feature>: user stories, acceptance criteria, data shapes, aff
ected files, edge cases, and open questions. List your assumptions explicitly. Do not write implementation un
til I approve the plan.
```

Schema-first

```
Design the database schema and TypeScript types for <feature> first. Output the SQL migration and the shared
types file. List every table, column, constraint, and index, and flag any nullable field that should be requi
red. No feature code yet.
```

Test generation

```
Write tests for <module> before the implementation using <test framework>. Cover the happy path, boundary val
ues, and the three most likely failure modes. Show me the tests, confirm they fail, then implement until they
 all pass.
```

Code review

```
Review this diff as a staff engineer. Flag correctness bugs, unhandled errors, N+1 queries, and anything that
 breaks the offline-first contract. For each issue give file, line, why it's wrong, and the minimal fix. Rank
 by severity, and say if any hunk is fine as-is.
```

Debugging / self-fix

```
Here is the full stack trace, the exact command I ran, and the last 40 lines of logs. Diagnose the root cause
 before proposing a fix: give me your top two hypotheses, tell me which files to check to confirm, then patch
 it and explain what was wrong.
```

Unlimited Opus 4.8 in Antigravity — a practical hack

Hit your usage limit? Antigravity resets it per Google account — sign in with a different Gmail and you're back to full Opus 4.8. The catch: switching accounts clears your session context. Before you switch, save what matters — copy your `plan.md`, key decisions, and the current chat into the repo (a `docs/` note) so the fresh session picks up exactly where you left off.

SECTION · 07

UI/UX Expectations

Design is not decoration — it is how the product works. Aim for the quiet confidence of Linear, Stripe, and Vercel: restrained, fast, obvious.

Space on a grid, never by eye. Snap every margin, padding, and gap to a 4px base with 8px rhythm. If a value is 13px or 17px, the scale is wrong, not the exception.

Hierarchy by weight, not by boxes. One primary action, one h1, everything else steps down. Rank things with size, weight, and color; reach for borders and fills last.

Restraint is the aesthetic. 3 neutral grays, one accent, one radius, one font family. Every extra color, shadow, or weight is a decision you're forcing on the user.

Motion only on state change. Animate enter/exit, expand/collapse, and position — 150-250ms, ease-out. Over 300ms reads as lag; honor prefers-reduced-motion and kill it entirely.

Design all four states first. Empty, loading, error, and success before the happy path. Skeletons over spinners; errors name the fix, never 'Something went wrong'.

Contrast that clears AA. Body text >=4.5:1, large text and UI/icons >=3:1. Placeholder-gray is not real content — if users must read it, it fails.

Keyboard is a first-class input. Every flow completes without a mouse: Tab order follows visual order, Cmd+K palette, Esc closes, Enter submits, and a visible focus ring on every control.

Dark mode is designed, not inverted. Elevate surfaces with lighter grays instead of shadows, desaturate accents, and cap 'white' text near 87% opacity to stop halation on pure black.

Consistency through tokens. One button, one input, one card driven by shared tokens. If two elements are 90% alike, make them identical or make the difference obvious.

Acknowledge input under 100ms. Hover, active, focus, and disabled states on every control, with instant press feedback. Use optimistic UI for reversible actions instead of a blocking spinner.

Hard specs you can copy

Token	Value
Spacing scale (4px base)	4, 8, 12, 16, 24, 32, 48, 64px — 8px is the default rhythm unit
Type scale	12, 14, 16, 20, 24, 32, 48px; body 16px, line-height 1.5, headings 1.2
Base radius	8px default — 6px inputs, 12px cards/modals, 9999px pills
Min tap target	44x44px (Apple HIG); 24x24px absolute floor (WCAG 2.2)
Contrast (WCAG AA)	4.5:1 body text, 3:1 large text (>=24px or 18.66px bold) and UI/icons
Max content width	~680px for prose (60-75ch), ~1280px app shell
Motion durations	150ms micro (hover/toggle), 200-250ms transitions, 300ms hard cap; ease cubic-bezier(0.4, 0, 0.2, 1)
Font pairing	Inter or Geist Sans for UI/body + Geist Mono / JetBrains Mono for numbers, code, and data
Focus ring	2px accent outline + 2px offset, visible on keyboard focus (:focus-visible), never removed
Responsive breakpoints	640 / 768 / 1024 / 1280px — design mobile-first, layout up
Elevation / shadow	Low-opacity, layered (e.g. 0 1px 2px + 0 4px 12px at 4-8% black); in dark mode raise surface lightness instead

SECTION · 08

Functional Requirements

These are the table stakes. If any category is missing, the product reads as a demo, not software. Cover them before you add anything clever.

Auth & Access

- Email+password signup hashing with Argon2id (or bcrypt cost ≥12); require email verification before granting write access.
- Store sessions in httpOnly, Secure, SameSite=Lax cookies and rotate the session ID on login and every privilege change to kill fixation.
- Password reset via single-use token, hashed at rest, 15–30 min TTL, invalidated on first use — never email the plaintext password.
- Enforce RBAC server-side on every route (owner/admin/member/viewer) in middleware; never trust a role sent from the client.
- Rate-limit login and reset to ~5 attempts/15 min per IP+account with exponential backoff to blunt credential stuffing.
- Wire OAuth (Google/GitHub) through a vetted lib like Auth.js/NextAuth — don't hand-roll the token exchange.

Data & CRUD

- Full create/read/update/delete on core entities with server-generated IDs and created_at/updated_at timestamps.
- Validate on both sides with one shared Zod schema so the browser and the API reject the exact same bad input.
- Use optimistic UI for high-success mutations (toggles, reorders) and roll back with an error toast on failure.
- Show explicit pending state — disable the submit button and show a spinner — so double-submits are structurally impossible.
- Soft-delete with deleted_at where recovery matters; define cascade rules explicitly instead of inheriting them by accident.
- Return the mutated record from the write so the client reconciles state without a second GET round-trip.

Finding Data

- Server-side search with debounced input (~300ms) backed by a trigram or full-text index — not client-side array filtering.
- Combine filters with clear AND semantics and mirror them into the URL query string so state is shareable and back-button works.
- Sort on indexed columns ascending/descending with a stable secondary sort on id to stop page jitter.
- Use cursor/keyset pagination for large sets (offset degrades past ~10k rows); default page size 25, hard-capped at 100.
- Distinguish 'no matches for this filter' from 'no data yet' and give the former a one-click reset.

State Coverage

- Every async view resolves to one of four states — loading, empty, error, success — with no blank flash in between.
- Render skeletons that match the final layout to avoid CLS; block interaction on the affected region, don't freeze the page.
- Make errors actionable: friendly message plus a retry button client-side, full stack trace logged server-side.
- Give empty states a primary CTA ('Create your first X') so the user never hits a dead end.
- Toast every mutation outcome, success and failure, with undo where reversible and ~4s auto-dismiss on success.

- Wrap routes in a 404 and a caught error boundary so a thrown render never becomes a white screen.

Trust & Safety

- Escape all rendered user input via framework auto-escaping; never pass raw user content to dangerouslySetInnerHTML.
- Authorize every mutation at the row level — verify the actor owns the resource, not merely that they're logged in.
- Rate-limit sensitive routes (auth, reset, export, webhooks) with a token bucket and return 429 + Retry-After.
- Keep secrets server-side only; nothing behind a NEXT_PUBLIC_ prefix may be a key — grep the client bundle to confirm.
- Use parameterized queries or an ORM everywhere to kill SQL injection, and allow-list uploads by MIME type and size.
- Set CSP, HSTS, and X-Content-Type-Options headers and enforce CSRF protection on all state-changing requests.

Beyond Basics

- CSV/PDF export of any table view, streamed for large sets so the request survives past the ~30s gateway timeout.
- Bulk actions via row selection with select-all-across-pages and a confirm step gating destructive operations.
- Keyboard shortcuts — Cmd/Ctrl+K command palette, j/k row nav, / to focus search — backed by a discoverable cheat sheet.
- Responsive from 320px up: touch targets ≥44px, zero horizontal scroll, tables collapsing to cards on mobile.
- Accessibility to WCAG 2.1 AA — semantic HTML, modal focus traps, ARIA labels, 4.5:1 contrast, full keyboard operability.
- Immutable audit/activity log capturing who changed what and when, queryable per entity.

SECTION · 09

Tech Stack

Use what you are fastest and most confident in — you are judged on the result, not the logos. If you have no strong preference, this modern, job-relevant stack is a safe default:

Layer	Recommended default
Framework	Next.js (App Router) or Remix — SSR, routing, API routes in one repo
Language	TypeScript in strict mode — no any, ever
UI	React + Tailwind CSS; shadcn/ui for accessible primitives
Database	PostgreSQL via Supabase / Neon, or SQLite for simple apps
ORM / Data	Prisma or Drizzle — typed queries, migrations in git
Auth	Auth.js (NextAuth), Clerk, or Supabase Auth — never hand-roll crypto
Validation	Zod on every boundary (forms, API, env)
Hosting	Vercel or Netlify (frontend) + managed Postgres
State/Data	TanStack Query or server actions; avoid global state until you need it
Tooling	ESLint + Prettier, Vitest/Playwright for tests, GitHub Actions CI

The only hard rules
TypeScript strict. A real database (not just `localStorage`). Real auth. Server-side validation. Secrets in env vars, never in the repo.

SECTION · 10

GitHub Repository Structure

A reviewer forms an opinion in the first ten seconds of opening your repo. Make the structure obvious and the root clean. A sensible layout for a Next.js app:

```
my-product/
├── .github/
│   ├── workflows/ci.yml          # lint + typecheck + test on every push
│   ├── ISSUE_TEMPLATE/            # bug_report.md, feature_request.md
│   └── PULL_REQUEST_TEMPLATE.md
├── docs/                         # architecture, screenshots, decisions
│   ├── architecture.md
│   └── screenshots/
├── prisma/                       # schema.prisma + migrations (committed)
├── public/                       # static assets, og-image.png, favicon
├── src/
│   ├── app/                      # routes (App Router)
│   ├── components/               # reusable UI (ui/ + feature components)
│   ├── lib/                      # db client, auth, utils, validators
│   ├── server/                   # server actions / API logic
│   └── types/
├── tests/                        # unit + e2e
├── .env.example                  # every var, with safe dummy values
├── .gitignore                    # .env, node_modules, .next
├── CHANGELOG.md
├── CONTRIBUTING.md
├── LICENSE
└── README.md
```

- **Commit `.env.example`, never `.env`.** The example documents every variable a cloner needs.
- **Keep the root tidy** — config files only. Source lives in `src/`.
- **Migrations are code.** Commit them so the schema is reproducible.
- **Add topics + a description + the live URL** to the repo header on GitHub.

SECTION · 11

README Template

The README is your product's front door and the single most-read file you will write. It must let a stranger understand, run, and evaluate the project in minutes. Fill in this skeleton:

```
# Product Name
> One sharp sentence: what it does and who it's for.

![Hero screenshot](docs/screenshots/hero.png)

[![CI](badge)]  [![License: MIT](badge)]  **Live demo → https://your-app.com**

##   Features
- Bullet the 5–8 things it actually does (verbs, not adjectives).

##   Tech Stack
Next.js · TypeScript · PostgreSQL (Prisma) · Tailwind · Auth.js · Vercel

##   Quick Start
```bash
git clone https://github.com/you/product && cd product
cp .env.example .env         # then fill in values
npm install
npm run db:migrate && npm run db:seed
npm run dev                  # http://localhost:3000
```

Environment Variables
Variable	Description
DATABASE_URL	Postgres connection string
AUTH_SECRET	Session signing secret

Architecture
Short paragraph + link to docs/architecture.md. One diagram beats a page of prose.

Testing
```bash
npm run test        # unit
npm run test:e2e    # playwright
```

Roadmap
- [ ] Shipped thing   - [ ] Next thing

Screenshots
Grid of the core flows.

License
MIT — see LICENSE.
```

Demo credentials
If the app has auth, put a read-only demo login in the README (`demo@demo.com / demo1234`). Reviewers will not sign up — make it zero-friction to see your work.

SECTION · 12

Documentation Requirements

Documentation is proof you can think clearly, not an afterthought. Write it as you build, while the decisions are fresh. Minimum viable docs:

README.md

- Covered above — the gateway. Screenshots, quick start, env, demo login.

docs/architecture.md

- A diagram of the data model (tables + relationships).
- How auth and authorization work in one paragraph.

- Any non-obvious decision, and the trade-off you made.

API / data documentation

- Document each endpoint or server action: method, path, inputs, output, auth required.
- For REST, a short table; for GraphQL, the schema; consider an OpenAPI file if it fits.

Inline & decisions

- Comment the *why*, never the *what*. Good names remove most comments.
- Keep a lightweight CHANGELOG so history reads as a story of progress.

> **Document as you build**
> Writing docs at the end means reconstructing decisions you've forgotten. A five-line note the moment you make a choice is worth an hour of archaeology later.

SECTION · 13

Deployment — Vercel or Netlify

An undeployed project does not exist. A live URL is non-negotiable — it is the first thing a reviewer opens. Both platforms have generous free tiers and deploy from GitHub in minutes.

Vercel (recommended for Next.js)

- Push your repo to GitHub, then **Import Project** at vercel.com — it auto-detects Next.js.
- Add every variable from `.env.example` under **Settings** → **Environment Variables**.
- Provision Postgres (Vercel Postgres, Neon, or Supabase) and set `DATABASE_URL`.
- Run migrations against the production DB (a `postinstall` or a one-off command).
- Every push to `main` auto-deploys; every PR gets a preview URL.

Netlify (great for Remix / static + functions)

- **Add new site** → **Import from Git**; set build command and publish directory.
- Configure env vars under **Site settings** → **Environment variables**.
- Use Netlify Functions or Edge Functions for server logic.

Before you call it deployed

- ☐ The live URL loads with **no console errors** and no broken images.
- ☐ Auth works end to end on production, not just localhost.
- ☐ A fresh visitor can sign up or use the demo login and complete the core flow.
- ☐ No secret keys leaked to the client bundle (check the network tab).
- ☐ The custom OG image renders when you paste the URL into Slack/Twitter.

SECTION · 14

SEO Requirements

A product nobody can find is a product nobody uses. Your landing page and docs must be technically discoverable and fast. Treat this as engineering, not marketing.

Technical Foundations

- Use semantic HTML5 landmarks: exactly one <h1> and one <main> per route, plus <nav>/<article>/<footer> — no div-soup.
- Unique <title> per route, 50-60 chars with the primary keyword front-loaded; meta description 150-160 chars written to earn the click.

- Emit <link rel="canonical"> with an absolute URL on every page to collapse trailing-slash and query-param duplicates.
- Ship /robots.txt that allows crawl, blocks /api and preview deploys, and points to the absolute sitemap URL.
- Generate sitemap.xml at build time (app/sitemap.ts or next-sitemap) with <lastmod>, then submit in Search Console and Bing Webmaster.
- Enforce lowercase kebab-case paths (/docs/getting-started), 301 the trailing-slash variant, and drop .html and query-string routing.

Social & Sharing

- Set full OG tags per route: og:title, og:description, og:url, og:type, og:site_name, og:image — all with absolute URLs.
- Add Twitter card tags: twitter:card=summary_large_image, twitter:title, twitter:description, twitter:image.
- Render a 1200x630 OG image under 8MB via Next ImageResponse (opengraph-image.tsx) so each page gets its own.
- Set og:image:width/height and og:image:alt so scrapers skip the re-fetch and stay accessible.
- Validate in the Facebook Sharing Debugger, LinkedIn Post Inspector, and Twitter Card Validator before launch — a local screenshot is not proof.

Structured Data

- Emit SoftwareApplication JSON-LD on the landing page: name, applicationCategory, operatingSystem: Web, offers (price 0), and aggregateRating only if reviews are real.
- Mark the FAQ block with FAQPage JSON-LD — question/answer text must match the visible copy verbatim or Google strips the rich result.
- Add BreadcrumbList JSON-LD on docs pages that mirrors the on-screen breadcrumb trail exactly.
- Inject JSON-LD as server-rendered <script type="application/ld+json">, never client-injected after hydration.
- Validate every block in the Google Rich Results Test and schema.org validator — zero errors and zero warnings before merge.

Performance & Core Web Vitals

- Hit p75 field thresholds — LCP <2.5s, INP <200ms, CLS <0.1 — verified in CrUX/PageSpeed, not just a lab run.
- Score Lighthouse >=90 across all four categories (Performance, Accessibility, Best Practices, SEO) and wire it into CI to block regressions.
- Serve images as AVIF/WebP through next/image with explicit width/height to reserve the layout box and kill CLS.
- Lazy-load below-the-fold images with loading="lazy" and mark the LCP hero priority so it preloads.
- Self-host Inter via next/font with font-display: swap — no render-blocking Google Fonts request, no FOIT.
- Preload the LCP asset, defer non-critical JS, and measure INP in production with the web-vitals library.

Content & Discovery

- Write a real landing page: a specific H1 value prop, feature sections, product screenshots, and one primary CTA — no lorem, no placeholder.
- Ship crawlable static docs pages (getting-started, features, deploy), each targeting a single search intent.
- Add an FAQ answering real user questions (sync, offline, pricing) that feeds both the FAQPage schema and AI Overviews.
- Make the README the repo's front door: badges, one-line pitch, a demo GIF, install steps, and a live-site link.
- Cross-link landing, docs, and FAQ with descriptive anchor text so crawlers and users flow between pages.
- Link the app and repo bidirectionally (README to live site, site footer to GitHub) to consolidate authority.

SECTION · 15

Open Source Requirements

Open-sourcing well is a skill of its own. These files signal that you understand how real projects are run and how strangers collaborate.

LICENSE

Add a real license — **MIT** is the safe, permissive default. Without one, your code is legally 'all rights reserved' and nobody can use it.

```
MIT License

Copyright (c) 2025 Your Name

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software... (full text — generate via choosealicense.com)
```

CONTRIBUTING.md

- How to set up the project locally (link to the README quick start).
- Branch naming, commit style, and how to open a PR.
- How to run tests and lint before pushing.

CHANGELOG.md — Keep a Changelog format

```
# Changelog
All notable changes to this project are documented here.

## [1.0.0] - 2025-06-01
### Added
- Authentication, dashboard, and CRUD for core entity.
### Fixed
- Timezone bug in the weekly report.
```

Conventional Commits & versioning

- Write commits like `feat:`, `fix:`, `docs:`, `refactor:` — small and meaningful.
- Tag a release (`v1.0.0`) when the core is done; follow semantic versioning.
- Add issue templates and a PR template in `.github/` so collaboration has rails.

> **Commit hygiene is a tell**
> A history of *'fix', 'fix2', 'asdf', 'final-final'* screams junior. Small, well-labelled commits that each do one thing tell a reviewer you work with discipline.

SECTION · 16

Portfolio Requirements

Done right, this single project can carry your portfolio — for many developers it becomes the strongest thing they can point to. Package it so it sells your skill on its own, long after the trial is over.

A written case study

- **Problem** — what real pain does this solve, and for whom?
- **Approach** — key decisions, the data model, the trade-offs you accepted.
- **Result** — screenshots, the live link, what you'd build next.
- **What you learned** — the honest, specific version.

Assets to produce

- A 60–90 second screen-recorded demo (Loom) walking the core flow.
- 3–5 clean screenshots of the best screens for your README and portfolio.
- A one-paragraph pitch you can paste into LinkedIn or a job application.

Where it lives

- Pinned on your GitHub profile with topics and the live URL.
- A card on your personal portfolio site linking to demo + repo + case study.
- A short 'show, don't tell' post — the build, one hard problem, the result.

THE EDGE · 17

Pro Hacks & Tricks

The small, non-obvious moves that separate people who finish clean from people who thrash. Internalize these.

Plan Before You Build

→ Generate the DB schema and seed data first — every feature inherits its shape, so schema churn later is the most expensive kind.

→ Write the README's "what it does" paragraph before any code — if you can't describe the product in three sentences, the scope is still lying to you.

→ List every screen with its empty, loading, and error states in one flat file before styling anything — the states you skip in planning are the bugs you ship.

→ Draw the data flow source → store → render on a single page and name each boundary — most rewrites are just a boundary you discovered too late.

→ Timebox risky spikes to a day and throw the code away — proving an integration works is planning; keeping the throwaway is debt you signed for.

AI-Assisted Coding

→ Paste the raw error, full stack trace, and failing file into the prompt — Claude debugging your paraphrase is guessing; from the trace it's diagnosing.

→ Feed it your types and interfaces before asking for logic — models hallucinate far less against a concrete contract than against prose.

→ Read every generated diff line-by-line before accepting — you own the production bug, not the model, so review it like a PR from a junior.

→ Ask for the failing test first, then the implementation — a spec you wrote beats code you eyeballed as "looks right."

→ Scope each prompt to one file or one function — a "build the whole feature" request returns 400 lines you won't actually read.

Ship & Deploy

→ Deploy a hello-world to production on day one — the pipeline is a dependency, and finding it broken at feature-complete costs you the launch.

→ Put every secret in platform env vars, never the repo — one committed key means rotating credentials at 2am and a permanent line in git history.

→ Wire per-branch preview deploys so every PR has a live URL — reviewing a running app catches what reviewing a diff never will.

→ Make CI fail on type and lint errors, not just warn — a green build that ships broken types is worse than having no CI at all.

→ Run the production build locally before pushing — `next build` surfaces static-export and SSR breakage that `next dev` silently tolerates.

Performance

→ Profile with the Network tab and Lighthouse before optimizing anything — the slow thing is almost never the thing you assumed.

→ Serve images as WebP/AVIF with explicit width and height — unsized images cause layout shift, and CLS is the cheapest score you'll ever recover.

→ Lazy-load routes and heavy below-the-fold components — the user pays for every KB in the initial bundle whether they scroll or not.

→ Debounce anything firing on scroll, resize, or keystroke — a 200ms debounce on search turns 40 renders into two.

→ Memoize the hot 5% you measured, not every component — blanket memoization adds complexity to code that was never the bottleneck.

Git & Workflow

→ Commit in small working increments with imperative messages — a 600-line commit called "updates" is un-reviewable and un-revertable.

→ Branch per feature and never push straight to main — a broken main blocks every deploy and every teammate at the same moment.

→ Write PR descriptions as "what changed and why" — the reviewer's speed is capped by how fast they can reconstruct your intent.

→ Rebase on main before opening the PR — resolving conflicts on your own branch is cheaper than fighting them in the merge queue.

→ Commit a .gitignore for node_modules, .env, and build output before commit one — junk in history is nearly impossible to fully scrub later.

Polish & Detail

→ Design the empty state before the populated one — users hit "zero items" first, and a blank screen reads as broken.

→ Use loading skeletons over spinners for content with a known shape — skeletons feel faster because they promise exactly what's coming.

→ Give every interactive element hover, focus, active, and disabled states — the gap between a demo and a product lives in those four.

→ Test keyboard-only and at mobile width before calling it done — the bugs invisible with a mouse on a 27-inch monitor are the first ones users report.

→ Reserve space for async content so nothing jumps on load — layout that shifts as it renders is the fastest way to feel amateur.

SECTION · 18

Common Mistakes

Every one of these is common, avoidable, and costs real points. Read them once now and once again the night before you submit.

The mistake	Do this instead
Ships a demo where data vanishes on refresh - no persistence, no auth, just React state.	Wire real persistence (Supabase/localStorage) and gate writes behind login; a stateless mock is a wireframe, not a product.
Commits .env or API keys straight into the repo.	Rotate the leaked key immediately, add .env to .gitignore, move secrets to platform env vars, and scrub history with git filter-repo before pushing.
Leaves the framework's default README (`npm run dev` and a Next.js logo).	Replace it with a pitch, screenshot/GIF, live URL, and setup steps you re-ran on a fresh clone.

The mistake	Do this instead
One squashed 'initial commit' containing the entire project.	Commit in logical units as you build - reviewers read your git log to judge how you think, not just the final diff.
Only runs on localhost - never deployed, or the deploy 404s on refresh.	Deploy to Vercel/Netlify, open the live URL in incognito, and click every nav item and deep-link before submitting.
Zero meta tags or OG image, so shared links render a blank grey card.	Add per-route title/description, a 1200x630 OG image, and verify in the platform sharing debuggers.
Console is full of React key warnings, hydration errors, and failed 404 fetches.	Open DevTools and drive it to zero errors and warnings; treat a red console as a failing test that blocks submission.
No empty, loading, or error states - blank screen while fetching, white crash on failure.	Design all three states explicitly and wrap every async call in try/catch with a real user-facing message.
Layout breaks below 400px - horizontal scroll and overlapping text on mobile.	Build mobile-first and test at 375px in DevTools device mode; no horizontal scrollbar, ever.
Leaves AI attribution, TODOs, commented-out code, and unused scaffolding in the tree.	Strip generated comments and dead files, and own the codebase as if you typed every line - because you're accountable for it.

BEFORE YOU SUBMIT · 19

Submission Checklist

Do not submit until every box is checked. This is the exact list a reviewer runs.

Product

- ☐ Live URL loads fast with zero console errors.
- ☐ Real auth (signup/login) works on production.
- ☐ Core CRUD works: create, read, update, delete — verified by hand.
- ☐ Search / filter / sort / pagination behave under real data volume.
- ☐ Every screen handles loading, empty, and error states.
- ☐ Responsive on mobile; keyboard-navigable; passes basic a11y checks.

Code & Repo

- ☐ TypeScript strict, no errors; lint and tests pass in CI.
- ☐ README with screenshots, quick start, env table, and demo login.
- ☐ LICENSE, CONTRIBUTING, CHANGELOG, and `.env.example` present.
- ☐ Clean commit history and a tagged `v1.0.0` release.
- ☐ No secrets committed; `.env` is gitignored.

Discoverability & Portfolio

- ☐ Meta tags, OG image, sitemap, and robots.txt in place.
- ☐ Lighthouse ≥ 90 on Performance, Accessibility, Best Practices, SEO.
- ☐ Demo video recorded; case study written.

SECTION · 20

Evaluation Criteria

Every submission is scored on the rubric below. Weights tell you where to spend your last free hour. Nothing here rewards scope for its own sake.

Criterion	Weight	What an excellent score looks like
Product Quality & Functionality	20%	A stranger completes the core job-to-be-done in under 60 seconds with zero dead ends; happy path, empty state, bad input, and offline all handled without a single uncaught console error.
UI/UX Craft	15%	One 4/8px spacing scale, one type scale, WCAG AA contrast, keyboard-navigable, with designed loading/skeleton and error states; Lighthouse a11y >= 95 and it reads as intentional, not Bootstrap-default.
Code Quality & Architecture	15%	TypeScript strict with zero `any`, clear module boundaries, no 500-line God components, no dead code; a senior can add a feature in one sitting without asking where anything lives.
Deployment & Reliability	12%	Live public HTTPS URL with a custom domain, CI running typecheck + build on every push, env vars documented, and it still works after a hard refresh in incognito.
Documentation	10%	README opens with a one-line pitch, screenshot/GIF, and live demo link above the fold, then clean-clone setup steps you actually re-ran, an env-var table, and short architecture notes.
GitHub Professionalism	10%	Atomic, conventionally-named commits that tell a build story, no secrets anywhere in history, correct .gitignore, a real LICENSE - not one 4000-line 'final final' dump.
SEO & Discoverability	10%	Unique title/meta-description per route, OpenGraph + Twitter card image, semantic single H1, sitemap.xml + robots.txt, JSON-LD schema, and a Lighthouse SEO score of 100.
Originality & Attention to Detail	8%	Solves a real, specific problem with an opinionated take, finished off with custom favicon, styled 404, and sensible defaults - the details that prove you dogfooded your own product.

GO FURTHER · 21

Bonus Points

Optional, but each one moves you from 'good candidate' to 'obvious hire'. Pick a few that fit your product — do not bolt on all of them.

- ★ **Automated tests that matter** — an e2e test of the critical path, not coverage theatre.
- ★ **CI/CD** — GitHub Actions running lint, typecheck, and tests on every PR.
- ★ **Real-time or optimistic UI** — the product feels instant.
- ★ **Role-based access** — admin vs. member with enforced permissions.
- ★ **Dark mode** done properly (system-aware, no flash).
- ★ **An AI feature** that genuinely helps the user, not a gimmick.
- ★ **Analytics dashboard** with real charts over your own data.
- ★ **Accessibility above the bar** — full keyboard support, focus states, ARIA where needed.
- ★ **A custom domain** and a polished 404 page.
- ★ **A short architecture write-up** explaining one hard decision.

HOW TO SUBMIT · 22

Final Submission

Every deliverable below must be live and public before you submit. Make it effortless to evaluate.

Deliverable	What to include
1 · Live app	Deployed to a real Vercel or Netlify subdomain — they give you one free (e.g. `your-app.vercel.app`). Include the demo login if the app has auth.
2 · Public GitHub repo	Public repo with a proper README: what the product does, how to run and **use** the software, screenshots, env setup, and a credit to the Digital Heroes trial. Add a LICENSE and a tagged release.
3 · Demo video	A 60–90 second Loom walking through the core flow.
4 · Case study	A short write-up: problem, approach, result, and what you learned.

Worth more than the trial: build this well and it becomes one of the strongest pieces in your portfolio — a real, deployed, open-source product you can show any employer for years.

Then reach out — this is how you get selected

Once everything above is live, message **Shreyansh Singh** on **LinkedIn** (or **Instagram**) with your links. That is how you enter review — we go through your work personally and follow up with feedback and your next steps. A finished project nobody is told about can't move you forward, so don't skip this.

One last standard to hold yourself to

Before you hit send, ask the only question that matters: **"Would a staff engineer be impressed by this — or just tolerate it?"** If the honest answer is 'tolerate', you have found your next hour of work. Ship the version you'd be proud to put your name on.

Now go build something real. — Prasun Anand, Founder, Digital Heroes